建议使用 python 虚拟环境来运行，若使用的是 jupyter notebook，可以按下面步骤构建一个虚拟环境： 1、打开 cmd 控制台，运行下面语句

```
conda create -n <虚拟环境名称>
```

2、激活刚刚创建的环境

```
conda activate <虚拟环境名称>
```

3、下载 ipykernel库

```
conda install ipykernel
```

4、将该虚拟环境写入notebook的kernel中

```
python -m ipykernel install --user --name <创建的虚拟环境名称> --
display-name <jupyter notebook中显示的名称>
```

5、在 jupyter notebook 中即可切换为 python 虚拟环境

在运行代码前需要保证已安装 python 及项目所需要的包

```
# 安装核心依赖
pip install requests>=2.31.0
pip install tavily-python>=0.3.0
pip install openai>=1.0.0

# 可选：安装其他常用包
pip install python-dotenv>=1.0.0
```

In [1]:
```
import requests
import os
import re
from openai import OpenAI
from tavily import TavilyClient
from dotenv import load_dotenv

# 加载环境变量
load_dotenv()

# 配置API密钥，获取名称分别为 API_KEY、BASE_URL、MODEL_ID、TAVILY_API_KEY 的环境变
# 【注】：需要根据根目录下 .env 设置的名称填写，或者直接把等号后面的改成字符串
API_KEY = os.getenv("OPENAI_API_KEY")
BASE_URL = os.getenv("OPENAI_BASE_URL")
MODEL_ID = os.getenv("MODEL_NAME")
TAVILY_API_KEY = os.getenv("TAVILY_API_KEY")

os.environ['TAVILY_API_KEY'] = TAVILY_API_KEY

# 系统提示词
AGENT_SYSTEM_PROMPT = """
你是一个智能旅行助手。你的任务是分析用户的请求，并使用可用工具一步步地解决问题。
```

```
# 可用工具:
- `get_weather(city: str)`: 查询指定城市的实时天气。
- `get_attraction(city: str, weather: str)`: 根据城市和天气搜索推荐的旅游景点。

# 输出格式要求:
你的每次回复必须严格遵循以下格式，包含一对Thought和Action:

Thought: [你的思考过程和下一步计划]
Action: [你要执行的具体行动]

Action的格式必须是以下之一:
1. 调用工具: function_name(arg_name="arg_value")
2. 结束任务: Finish[最终答案]

# 重要提示:
- 每次只输出一对Thought-Action
- Action必须在同一行，不要换行
- 当收集到足够信息可以回答用户问题时，必须使用 Action: Finish[最终答案] 格式结束

请开始吧!
"""
```

In [2]:

```python
def get_weather(city: str) -> str:
    """
    通过调用 wttr.in API 查询真实的天气信息。
    wttr.in: https://github.com/chubin/wttr.in
    """
    # API端点，我们请求JSON格式的数据
    url = f"https://wttr.in/{city}?format=j1"

    try:
        # 发起网络请求
        response = requests.get(url)
        # 检查响应状态码是否为200 (成功)
        response.raise_for_status()
        # 解析返回的JSON数据
        data = response.json()

        # 提取当前天气状况
        current_condition = data['current_condition'][0]
        weather_desc = current_condition['weatherDesc'][0]['value']
        temp_c = current_condition['temp_C']

        # 格式化成自然语言返回
        return f"{city}当前天气: {weather_desc}, 气温{temp_c}摄氏度"

    except requests.exceptions.RequestException as e:
        # 处理网络错误
        return f"错误: 查询天气时遇到网络问题 - {e}"
    except (KeyError, IndexError) as e:
        # 处理数据解析错误
        return f"错误: 解析天气数据失败，可能是城市名称无效 - {e}"

def get_attraction(city: str, weather: str) -> str:
    """
    根据城市和天气，使用Tavily Search API搜索并返回优化后的景点推荐。
    """
    api_key = os.environ.get("TAVILY_API_KEY")

    if not api_key:
```

```
        return "错误：未配置TAVILY_API_KEY。"

    # 初始化Tavily客户端
    tavily = TavilyClient(api_key=api_key)

    # 构造一个精确的查询
    query = f"'{city}' 在'{weather}'天气下最值得去的旅游景点推荐及理由"

    try:
        # 调用API，include_answer=True会返回一个综合性的回答
        response = tavily.search(query=query, search_depth="basic", include_answ

        # Tavily返回的结果已经非常干净，可以直接使用
        if response.get("answer"):
            return response["answer"]

        # 如果没有综合性回答，则格式化原始结果
        formatted_results = []
        for result in response.get("results", []):
            formatted_results.append(f"- {result['title']}: {result['content']}"

        if not formatted_results:
             return "抱歉，没有找到相关的旅游景点推荐。"

        return "根据搜索，为您找到以下信息：\n" + "\n".join(formatted_results)

    except Exception as e:
        return f"错误：执行Tavily搜索时出现问题 - {e}"

# 将所有工具函数放入一个字典，方便后续调用
available_tools = {
    "get_weather": get_weather,
    "get_attraction": get_attraction,
}
print("✅ 工具函数定义完成!")
```

✅ 工具函数定义完成!

In [3]:

```
class OpenAICompatibleClient:
    """
    一个用于调用任何兼容OpenAI接口的LLM服务的客户端。
    """
    def __init__(self, model: str, api_key: str, base_url: str):
        self.model = model
        self.client = OpenAI(api_key=api_key, base_url=base_url)

    def generate(self, prompt: str, system_prompt: str) -> str:
        """调用LLM API来生成回应。"""
        print("正在调用大语言模型...")
        try:
            messages = [
                {'role': 'system', 'content': system_prompt},
                {'role': 'user', 'content': prompt}
            ]
            response = self.client.chat.completions.create(
                model=self.model,
                messages=messages,
                stream=False
            )
            answer = response.choices[0].message.content
```

```
            print("大语言模型响应成功。")
            return answer
        except Exception as e:
            print(f"调用LLM API时发生错误: {e}")
            return "错误: 调用语言模型服务时出错。"

class TravelAssistant:
    """
    智能旅行助手类
    """
    def __init__(self):
        self.llm = OpenAICompatibleClient(
            model=MODEL_ID,
            api_key=API_KEY,
            base_url=BASE_URL
        )
        self.prompt_history = []

    def reset(self):
        """重置对话历史"""
        self.prompt_history = []

    def add_user_message(self, message: str):
        """添加用户消息到历史"""
        self.prompt_history.append(f"用户请求: {message}")

    def add_assistant_message(self, message: str):
        """添加助手消息到历史"""
        self.prompt_history.append(message)

    def add_observation(self, observation: str):
        """添加观察结果到历史"""
        self.prompt_history.append(f"Observation: {observation}")
print("✅ 智能助手类定义完成!")
```

✅ 智能助手类定义完成!

In [4]:

```
def display_conversation(history):
    """美观地显示对话历史"""
    print("\n" + "="*60)
    print("📝 对话历史")
    print("="*60)

    for i, message in enumerate(history, 1):
        if message.startswith("用户请求:"):
            print(f"\n👤 用户 [{i}]: {message[5:]}")
        elif message.startswith("Thought:"):
            print(f"\n🤔 思考 [{i}]: {message[8:].strip()}")
        elif message.startswith("Action:"):
            print(f"🛠️  行动 [{i}]: {message[7:].strip()}")
        elif message.startswith("Observation:"):
            print(f"📊 观察 [{i}]: {message[12:].strip()}")
        else:
            print(f"💬 消息 [{i}]: {message}")

    print("="*60 + "\n")

def parse_action(action_str):
    """解析行动字符串"""
    if action_str.startswith("Finish"):
```

```
        match = re.match(r"\w+\[(.*)\]", action_str)
        if match:
            return "finish", {"answer": match.group(1)}
        return "finish", {"answer": "任务完成"}

    tool_name_match = re.search(r"(\w+)\(", action_str)
    if not tool_name_match:
        return None, {}

    tool_name = tool_name_match.group(1)
    args_match = re.search(r"\((.*)\)", action_str)
    if args_match:
        args_str = args_match.group(1)
        kwargs = dict(re.findall(r'(\w+)="([^"]*)"', args_str))
    else:
        kwargs = {}

    return tool_name, kwargs
print("✅ 显示函数定义完成!")
```

✅ 显示函数定义完成!

In [5]:

```
def run_assistant(user_input, max_iterations=5, display=True):
    """
    运行旅行助手的主函数

    Args:
        user_input: 用户输入的问题
        max_iterations: 最大循环次数
        display: 是否显示对话历史

    Returns:
        tuple: (最终答案, 完整的对话历史)
    """
    assistant = TravelAssistant()
    assistant.add_user_message(user_input)

    if display:
        print(f"👤 用户输入: {user_input}")
        print("="*50)

    for i in range(max_iterations):
        if display:
            print(f"\n🔄 循环 {i+1}/{max_iterations}")

        # 构建完整prompt并调用LLM
        full_prompt = "\n".join(assistant.prompt_history)
        llm_output = assistant.llm.generate(full_prompt, AGENT_SYSTEM_PROMPT)
        # 模型可能会输出多余的Thought-Action，需要截断
        match = re.search(r'(Thought:.*?Action:.*?)(?=\n\s*(?:Thought:|Action:|O
        if match:
            truncated = match.group(1).strip()
            if truncated != llm_output.strip():
                llm_output = truncated
                print("⚠️ 已截断多余的 Thought-Action 对")

        assistant.add_assistant_message(llm_output)

        if display:
            print(f"🤖 模型输出:\n{llm_output}")
```

```
        # 解析行动
        action_match = re.search(r"Action: (.*)", llm_output, re.DOTALL)
        if not action_match:
            observation = "错误：未能解析到 Action 字段。请确保你的回复严格遵循 '
            observation_str = f"Observation: {observation}"
            print(f"{observation_str}\n" + "="*40)
            assistant.prompt_history.append(observation_str)
            continue

        action_str = action_match.group(1).strip()
        tool_name, kwargs = parse_action(action_str)

        # 处理完成行动
        if tool_name == "finish":
            final_answer = kwargs.get("answer", "任务完成")
            if display:
                print(f"🎉 任务完成!")
                print(f"📋 最终答案：{final_answer}")
            return final_answer, assistant.prompt_history

        # 处理工具调用
        if tool_name in available_tools:
            if display:
                print(f"🛠️  调用工具：{tool_name}({kwargs})")
            observation = available_tools[tool_name](**kwargs)
        else:
            observation = f"错误：未定义的工具 '{tool_name}'"

        # 记录观察结果
        if display:
            print(f"📊 观察结果：{observation}")
            print("="*50)

        assistant.add_observation(observation)

    # 如果达到最大循环次数仍未完成
    timeout_answer = "抱歉，经过多次尝试仍未完成您的请求。请尝试简化您的问题或稍后
    if display:
        print(f"⏰ 达到最大循环次数：{timeout_answer}")

    return timeout_answer, assistant.prompt_history
```

In [6]:

```
# 测试示例
def test_basic_example():
    """测试北京天气+景点推荐的示例"""
    print("🚀 开始测试北京天气+景点推荐示例")
    user_input = "你好，请帮我查询一下今天北京的天气，然后根据天气推荐一个合适的旅

    final_answer, history = run_assistant(user_input, display=True)

    print("\n" + "="*60)
    print("📊 测试完成!")
    print("="*60)
    print(f"最终答案：{final_answer}")

    # 显示完整对话历史
    display_conversation(history)

    return final_answer, history
```

```
# 运行测试示例
final_answer, history = test_basic_example()
```

🚀 开始测试北京天气+景点推荐示例
👤 用户输入：你好，请帮我查询一下今天北京的天气，然后根据天气推荐一个合适的旅游景点。
==========

🔄 循环 1/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 用户需要先了解北京今天的天气，再根据天气推荐景点。我先调用天气查询工具获取实时天气信息。
Action: get_weather(city="北京")
🛠️ 调用工具：get_weather({'city': '北京'})
📊 观察结果：北京当前天气：Sunny，气温16摄氏度
==========

🔄 循环 2/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 已获取北京今天天气为晴朗、气温16摄氏度。接下来需要根据“晴朗”天气查询适合游玩的景点。
Action: get_attraction(city="北京", weather="Sunny")
🛠️ 调用工具：get_attraction({'city': '北京', 'weather': 'Sunny'})
📊 观察结果：在北京的sunny天气下，推荐的旅游景点包括天安门广场和故宫。天安门广场是北京的中心地带，有严格的安全检查和长时间的排队，但一旦进入，可以感受到历史的厚重感。故宫，即紫禁城，是世界上最大的宫殿群，展现了明清两代的繁荣。这些景点不仅历史悠久，而且在北京市中心，交通便利。
==========

🔄 循环 3/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 已获取北京今天晴朗、16摄氏度的天气信息，并得到适合晴朗天气游玩的景点推荐，可以回答用户。
Action: Finish[北京今天天气晴朗，气温约16摄氏度，适合户外和观光游览；推荐天安门广场和故宫，二者位于市中心、交通便利且历史文化底蕴深厚，其中天安门广场可感受历史厚重感，故宫可欣赏明清宫殿建筑群。]
🎉 任务完成！
📋 最终答案：北京今天天气晴朗，气温约16摄氏度，适合户外和观光游览；推荐天安门广场和故宫，二者位于市中心、交通便利且历史文化底蕴深厚，其中天安门广场可感受历史厚重感，故宫可欣赏明清宫殿建筑群。

==========
📊 测试完成！
==========
最终答案：北京今天天气晴朗，气温约16摄氏度，适合户外和观光游览；推荐天安门广场和故宫，二者位于市中心、交通便利且历史文化底蕴深厚，其中天安门广场可感受历史厚重感，故宫可欣赏明清宫殿建筑群。

==========
📝 对话历史
==========

👤 用户 [1]: 你好，请帮我查询一下今天北京的天气，然后根据天气推荐一个合适的旅游景点。

🤔 思考 [2]: 用户需要先了解北京今天的天气，再根据天气推荐景点。我先调用天气查询工具获取实时天气信息。

Action: get_weather(city="北京")
📊 观察 [3]: 北京当前天气：Sunny，气温16摄氏度

🧐 思考 [4]: 已获取北京今天天气为晴朗、气温16摄氏度。接下来需要根据“晴朗”天气查询适合游玩的景点。
Action: get_attraction(city="北京", weather="Sunny")
📊 观察 [5]: 在北京的sunny天气下，推荐的旅游景点包括天安门广场和故宫。天安门广场是北京的中心地带，有严格的安全检查和长时间的排队，但一旦进入，可以感受到历史的厚重感。故宫，即紫禁城，是世界上最大的宫殿群，展现了明清两代的繁荣。这些景点不仅历史悠久，而且在北京市中心，交通便利。

🧐 思考 [6]: 已获取北京今天晴朗、16摄氏度的天气信息，并得到适合晴朗天气游玩的景点推荐，可以回答用户。
Action: Finish[北京今天天气晴朗，气温约16摄氏度，适合户外和观光游览；推荐天安门广场和故宫，二者位于市中心、交通便利且历史文化底蕴深厚，其中天安门广场可感受历史厚重感，故宫可欣赏明清宫殿建筑群。]
==

In [7]:
```python
def interactive_travel_assistant():
    """
    交互式旅行助手
    """
    print("🌍 欢迎使用智能旅行助手!")
    print("💡 您可以询问任何城市的天气和旅游景点推荐")
    print("❌ 输入 'quit' 或 '退出' 来结束对话\n")

    while True:
        user_input = input("👤 请输入您的问题: ").strip()

        if user_input.lower() in ['quit', '退出', 'exit']:
            print("👋 感谢使用智能旅行助手，再见!")
            break

        if not user_input:
            print("⚠️  请输入有效的问题")
            continue

        print("\n" + "="*50)
        print("🔄 正在处理您的请求...")

        final_answer, history = run_assistant(user_input, display=True)

        print("\n🎯 最终回答:")
        print("="*30)
        print(final_answer)
        print("="*30)

        # 询问是否显示完整对话历史
        show_history = input("\n📖 是否显示完整对话历史? (y/n): ").strip().lower
        if show_history in ['y', 'yes', '是']:
            display_conversation(history)

        print("\n" + "="*60)
        print("🔄 准备接受下一个问题...\n")

# 快速测试函数
def quick_test(city="上海"):
    """快速测试指定城市的天气和景点"""
    user_input = f"请帮我查询{city}的天气，并推荐适合的旅游景点"
```

```
    print(f"🚀 快速测试: {user_input}")
    final_answer, _ = run_assistant(user_input, display=True)
    return final_answer
```

In [8]:
```
# 主启动入口
if __name__ == "__main__":
    # 可以选择直接运行测试示例
    print("选择运行模式:")
    print("1. 运行测试示例 (北京)")
    print("2. 交互模式")
    print("3. 快速测试其他城市")

    choice = input("请输入选择 (1/2/3): ").strip()

    if choice == "1":
        test_basic_example()
    elif choice == "2":
        interactive_travel_assistant()
    elif choice == "3":
        city = input("请输入要测试的城市: ").strip() or "上海"
        quick_test(city)
    else:
        print("无效选择，运行测试示例...")
        test_basic_example()
```

```
选择运行模式:
1. 运行测试示例 (北京)
2. 交互模式
3. 快速测试其他城市
🌍 欢迎使用智能旅行助手!
💡 您可以询问任何城市的天气和旅游景点推荐
❌ 输入 'quit' 或 '退出' 来结束对话
```

```
==================================================
🔄 正在处理您的请求...
👤 用户输入：今日长沙适合去哪玩
==================================================

🔄 循环 1/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 用户想知道今天长沙适合去哪里玩，需要先查询长沙实时天气，再根据天气推荐景点。
Action: get_weather(city="长沙")
🛠️  调用工具：get_weather({'city': '长沙'})
📊 观察结果：长沙当前天气：Overcast ，气温20摄氏度
==================================================

🔄 循环 2/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 已获取长沙当前天气为阴天、20摄氏度，适合推荐室内外兼顾的景点。接下来查询阴天条件下长沙的推荐景点。
Action: get_attraction(city="长沙", weather="Overcast")
🛠️  调用工具：get_attraction({'city': '长沙', 'weather': 'Overcast'})
📊 观察结果：在长沙的阴天，最值得去的旅游景点包括湖南省博物馆、长沙博物馆和谢子龙影像艺术馆，这些室内文化场馆提供了“0落汤鸡风险”的文化体验。此外，湘江风光带，特别是橘子洲头和杜甫江阁，在细雨中的氤氲水面拍照出片率极高。历史老街如太平街、潮宗街和白果园巷的青石板路雨后泛光，也是拍照的好地方。
==================================================

🔄 循环 3/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 已获取长沙阴天、20℃的天气信息，并得到适合阴天的景点推荐，可以给出具体游玩建议。
Action: Finish[今日长沙为阴天、约20℃，适合以室内文化场馆为主、天气允许时搭配江边和老街漫步。推荐：1. 湖南省博物馆：室内看展，适合阴天；2. 长沙博物馆：文化体验且不受天气影响；3. 谢子龙影像艺术馆：室内拍照出片；4. 橘子洲头、杜甫江阁：阴天江面有氛围，适合拍照，但建议带伞；5. 太平街、潮宗街、白果园巷：雨后青石板路有味道，适合逛吃拍照。整体建议：优先安排博物馆和艺术馆，傍晚视天气去江边或老街，随身带伞。]
🎉 任务完成！
📋 最终答案：今日长沙为阴天、约20℃，适合以室内文化场馆为主、天气允许时搭配江边和老街漫步。推荐：1. 湖南省博物馆：室内看展，适合阴天；2. 长沙博物馆：文化体验且不受天气影响；3. 谢子龙影像艺术馆：室内拍照出片；4. 橘子洲头、杜甫江阁：阴天江面有氛围，适合拍照，但建议带伞；5. 太平街、潮宗街、白果园巷：雨后青石板路有味道，适合逛吃拍照。整体建议：优先安排博物馆和艺术馆，傍晚视天气去江边或老街，随身带伞。

🎯 最终回答：
==============================
今日长沙为阴天、约20℃，适合以室内文化场馆为主、天气允许时搭配江边和老街漫步。推荐：1. 湖南省博物馆：室内看展，适合阴天；2. 长沙博物馆：文化体验且不受天气影响；3. 谢子龙影像艺术馆：室内拍照出片；4. 橘子洲头、杜甫江阁：阴天江面有氛围，适合拍照，但建议带伞；5. 太平街、潮宗街、白果园巷：雨后青石板路有味道，适合逛吃拍照。整体建议：优先安排博物馆和艺术馆，傍晚视天气去江边或老街，随身带伞。
==============================
```

```
============================================================
📝 对话历史
============================================================

👤 用户 [1]:  今日长沙适合去哪玩

🤔 思考 [2]: 用户想知道今天长沙适合去哪里玩，需要先查询长沙实时天气，再根据天气推荐景点。
Action: get_weather(city="长沙")
📊 观察 [3]: 长沙当前天气：Overcast ，气温20摄氏度

🤔 思考 [4]: 已获取长沙当前天气为阴天、20摄氏度，适合推荐室内外兼顾的景点。接下来查询阴天条件下长沙的推荐景点。
Action: get_attraction(city="长沙", weather="Overcast")
📊 观察 [5]: 在长沙的阴天，最值得去的旅游景点包括湖南省博物馆、长沙博物馆和谢子龙影像艺术馆，这些室内文化场馆提供了“0落汤鸡风险”的文化体验。此外，湘江风光带，特别是橘子洲头和杜甫江阁，在细雨中的氤氲水面拍照出片率极高。历史老街如太平街、潮宗街和白果园巷的青石板路雨后泛光，也是拍照的好地方。

🤔 思考 [6]: 已获取长沙阴天、20℃的天气信息，并得到适合阴天的景点推荐，可以给出具体游玩建议。
Action: Finish[今日长沙为阴天、约20℃，适合以室内文化场馆为主、天气允许时搭配江边和老街漫步。推荐：1. 湖南省博物馆：室内看展，适合阴天；2. 长沙博物馆：文化体验且不受天气影响；3. 谢子龙影像艺术馆：室内拍照出片；4. 橘子洲头、杜甫江阁：阴天江面有氛围，适合拍照，但建议带伞；5. 太平街、潮宗街、白果园巷：雨后青石板路有味道，适合逛吃拍照。整体建议：优先安排博物馆和艺术馆，傍晚视天气去江边或老街，随身带伞。]
============================================================


============================================================
🔄 准备接受下一个问题...
```

==
🔄 正在处理您的请求...
👤 用户输入：洛阳呢
==

🔄 循环 1/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 用户询问“洛阳呢”，结合旅行助手场景，推测是想了解洛阳当前天气及适合游玩的景点。我先查询洛阳实时天气，再根据天气推荐景点。
Action: get_weather(city="洛阳")
🛠️ 调用工具：get_weather({'city': '洛阳'})
📊 观察结果：洛阳当前天气：Overcast ，气温18摄氏度
==

🔄 循环 2/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 已获取洛阳当前天气为阴天、18℃。接下来根据“阴天”这一天气条件查询洛阳适合游玩的景点推荐。
Action: get_attraction(city="洛阳", weather="Overcast")
🛠️ 调用工具：get_attraction({'city': '洛阳', 'weather': 'Overcast'})
📊 观察结果：在洛阳的阴天，白云山国家森林公园是一个推荐的旅游景点。公园位于洛阳市嵩县南部伏牛山腹地，全年开放时间为07:30-18:00，索道运营时间为08:00-18:00。门票价格为90元，推荐理由是“拾级而上，徜徉在金色的自然走廊中，满目皆是画，人在画中游。”此外，龙门石窟和王城公园也是值得一游的景点，但它们的具体推荐理由和开放时间在提供的信息中有所不同。
==

🔄 循环 3/5
正在调用大语言模型...
大语言模型响应成功。
🤖 模型输出：
Thought: 已获取洛阳实时天气和阴天景点推荐，信息足够回答用户。
Action: Finish[洛阳当前为阴天，气温约18℃。适合游玩白云山国家森林公园：位于洛阳市嵩县南部伏牛山腹地，全年开放07:30-18:00，索道运营08:00-18:00，门票90元；阴天漫步林间、登高赏景较舒适。此外，龙门石窟和王城公园也值得一游。]
🎉 任务完成！
📋 最终答案：洛阳当前为阴天，气温约18℃。适合游玩白云山国家森林公园：位于洛阳市嵩县南部伏牛山腹地，全年开放07:30-18:00，索道运营08:00-18:00，门票90元；阴天漫步林间、登高赏景较舒适。此外，龙门石窟和王城公园也值得一游。

🎯 最终回答：
==============================
洛阳当前为阴天，气温约18℃。适合游玩白云山国家森林公园：位于洛阳市嵩县南部伏牛山腹地，全年开放07:30-18:00，索道运营08:00-18:00，门票90元；阴天漫步林间、登高赏景较舒适。此外，龙门石窟和王城公园也值得一游。
==============================

```
============================================================
📝 对话历史
============================================================

👤 用户 [1]:  洛阳呢

🤔 思考 [2]: 用户询问“洛阳呢”，结合旅行助手场景，推测是想了解洛阳当前天气及适合游玩的景点。我先查询洛阳实时天气，再根据天气推荐景点。
Action: get_weather(city="洛阳")
📊 观察 [3]: 洛阳当前天气：Overcast ，气温18摄氏度

🤔 思考 [4]: 已获取洛阳当前天气为阴天、18℃。接下来根据“阴天”这一天气条件查询洛阳适合游玩的景点推荐。
Action: get_attraction(city="洛阳", weather="Overcast")
📊 观察 [5]: 在洛阳的阴天，白云山国家森林公园是一个推荐的旅游景点。公园位于洛阳市嵩县南部伏牛山腹地，全年开放时间为07:30-18:00，索道运营时间为08:00-18:00。门票价格为90元，推荐理由是“拾级而上，徜徉在金色的自然走廊中，满目皆是画，人在画中游。” 此外，龙门石窟和王城公园也是值得一游的景点，但它们的具体推荐理由和开放时间在提供的信息中有所不同。

🤔 思考 [6]: 已获取洛阳实时天气和阴天景点推荐，信息足够回答用户。
Action: Finish[洛阳当前为阴天，气温约18℃。适合游玩白云山国家森林公园：位于洛阳市嵩县南部伏牛山腹地，全年开放07:30-18:00，索道运营08:00-18:00，门票90元；阴天漫步林间、登高赏景较舒适。此外，龙门石窟和王城公园也值得一游。]
============================================================


============================================================
🔄 准备接受下一个问题...

👋 感谢使用智能旅行助手，再见!
```